
Modern C++ for Python Programmers

From Absolute Beginner to Systems, Graphics, and Game Development

2026

Contents

0.1	Modern C++ for Python Programmers	3
0.1.1	From Absolute Beginner to Systems, Graphics, and Game Development	3
0.1.2	How to Use This Book	3
0.1.3	Table of Contents	3
0.2	Part I – Foundations	4
0.2.1	What Does “Running a Program” Actually Mean?	4
0.2.2	How Python Runs Your Code	4
0.2.3	How C++ Runs Your Code	6
0.2.4	Your First C++ Program	8
0.2.5	Compiling: What the Flags Mean	9
0.2.6	What Is in the Compiled Binary?	10
0.2.7	Multi-File Programs	11
0.2.8	Reading Compiler Errors	11
0.2.9	Common Mistakes in This Chapter	13
0.2.10	Exercises	14
0.2.11	What IS a Variable?	15
0.2.12	How Integers Are Stored: Binary and Two’s Complement	16
0.2.13	The Fundamental Types	18
0.2.14	Fixed-Width Integer Types	20
0.2.15	Declaring and Initializing Variables	20
0.2.16	Memory Layout of Variables	22
0.2.17	auto – Type Deduction	22
0.2.18	const and constexpr	23
0.2.19	Type Conversion	24
0.2.20	Checking Sizes with sizeof	24
0.2.21	Common Mistakes in This Chapter	25
0.2.22	Exercises	26
0.2.23	Arithmetic Operators	28
0.2.24	Increment and Decrement Operators	29
0.2.25	Compound Assignment	30
0.2.26	Comparison Operators	30
0.2.27	Logical Operators	31
0.2.28	Bitwise Operators	32
0.2.29	Operator Precedence	34
0.2.30	The Ternary Operator	34
0.2.31	Common Mistakes in This Chapter	35
0.2.32	Exercises	36
0.2.33	if/else if/else	37
0.2.34	switch	38

0.2.35	while Loop	39
0.2.36	do-while Loop	40
0.2.37	for Loop: C-Style	40
0.2.38	Range-Based for Loop (C++11)	42
0.2.39	break and continue	43
0.2.40	Loop Execution Visualization	43
0.2.41	Common Mistakes in This Chapter	44
0.2.42	Exercises	46
0.2.43	What Happens When You Call a Function	48
0.2.44	Function Syntax	49
0.2.45	Declarations vs Definitions	49
0.2.46	Pass by Value: What Is Copied and Why It Matters	51
0.2.47	Default Parameters	52
0.2.48	Function Overloading	53
0.2.49	Compiler Messages for Function Errors	53
0.2.50	Common Mistakes in This Chapter	55
0.2.51	Exercises	56

0.1 Modern C++ for Python Programmers

0.1.1 From Absolute Beginner to Systems, Graphics, and Game Development

Target Standard: C++23 | **Prerequisites:** Comfortable Python, zero C++ assumed

0.1.2 How to Use This Book

Each chapter teaches one concept from scratch. Every concept is first shown in Python (which you know), then explained in C++, then built up with multiple examples, memory diagrams where helpful, common mistakes, and exercises with answers.

Work through Part I completely before moving on. The foundations – types, memory, functions – underpin everything else.

0.1.3 Table of Contents

Part I – Foundations 1. [The Compilation Model and Your First Program](#) 2. [Variables, Types, and the Static Type System](#) 3. [Operators and Expressions](#) 4. [Control Flow: Branching and Loops](#) 5. [Functions, Overloading, and Declarations vs Definitions](#)

Part II – Core C++ (the part that is not like Python) 6. [References – Aliases for Variables](#) 7. [Pointers and Memory Addresses](#) 8. [The Stack and the Heap](#) 9. [const Correctness](#) 10. [Arrays, `std::vector`, and `std::string`](#) 11. [Scope, Lifetime, and Organizing Code into Files](#)

Part III – Ownership and Memory Management 12. [RAII – The Core Idea That Replaces Garbage Collection](#) 13. [Dynamic Allocation: `new`, `delete`, and Why You Avoid Them](#) 14. [Smart Pointers: `unique\_ptr`, `shared\_ptr`, `weak\_ptr`](#) 15. [Move Semantics, lvalues and rvalues](#) 16. [The Rule of 0, 3, and 5](#)

Part IV – Object-Oriented Programming 17. [Classes, Objects, and Encapsulation](#) 18. [Constructors, Destructors, and Initialization](#) 19. [Inheritance and Composition](#) 20. [Virtual Functions and Polymorphism](#) 21. [Abstract Classes and Interfaces](#) 22. [Operator Overloading](#)

Part V – Generic Programming 23. [Function and Class Templates](#) 24. [Template Specialization and Variadic Templates](#) 25. [Concepts \(C++20\) – Compile-Time Duck Typing](#) 26. [An Introduction to Template Metaprogramming](#)

Part VI – The Standard Library 27. [Containers: `vector`, `map`, `set`, `array`, and `friends`](#) 28. [Iterators](#) 29. [Algorithms: `sort`, `find`, `transform`, and the rest](#) 30. [Lambdas and Function Objects](#) 31. [Ranges and Views \(C++20\)](#) 32. [Utility Types: `optional`, `variant`, `any`, `tuple`](#)

Part VII – Modern C++ (C++11 to C++23) 33. [`auto`, type deduction, and structured bindings](#) 34. [`constexpr` and compile-time computation](#) 35. [`std::format` and modern string handling](#) 36. [Coroutines and Generators](#) 37. [Modules \(C++20\)](#)

Part VIII – Performance 38. [Value vs Reference Semantics](#) 39. [How Memory Layout Affects Speed](#) 40. [Data-Oriented Design](#) 41. [Profiling and Flamegraphs](#)

Part IX – Concurrency 42. [Threads and `std::jthread`](#) 43. [Mutexes, Locks, and Race Conditions](#) 44. [Atomics and the C++ Memory Model](#) 45. [Async, Futures, and Tasks](#)

Part X – Graphics and Game Development 46. [The Math: vectors, matrices, quaternions](#) 47. [How the GPU Works](#) 48. [OpenGL Fundamentals](#) 49. [Moving to Vulkan](#) 50. [Game Loop, ECS Architecture, and Engine Design](#)

Part XI – Systems Programming 51. [The Machine: registers, memory, syscalls](#) 52. [Working with the OS and Linux APIs](#) 53. [Networking from the Ground Up](#) 54. [Where C++ Meets C, eBPF, and Go](#)

Appendices - [A. Setting Up Your Toolchain](#) - [B. Compiler Flags Reference](#) - [C. Python to C++ Cheat Sheet](#) - [D. Common Mistakes and How to Debug Them](#)

0.2 Part I – Foundations

Chapter 1: The Compilation Model and Your First Program

0.2.1 What Does “Running a Program” Actually Mean?

Before you write a single line of C++, you need to understand something Python completely hides: what a computer actually does to run your code.

When you type `python hello.py`, a lot of invisible machinery kicks in. When you compile and run a C++ program, that machinery becomes visible – and you control it. That visibility is both the source of C++’s power and the main source of beginner confusion.

0.2.2 How Python Runs Your Code

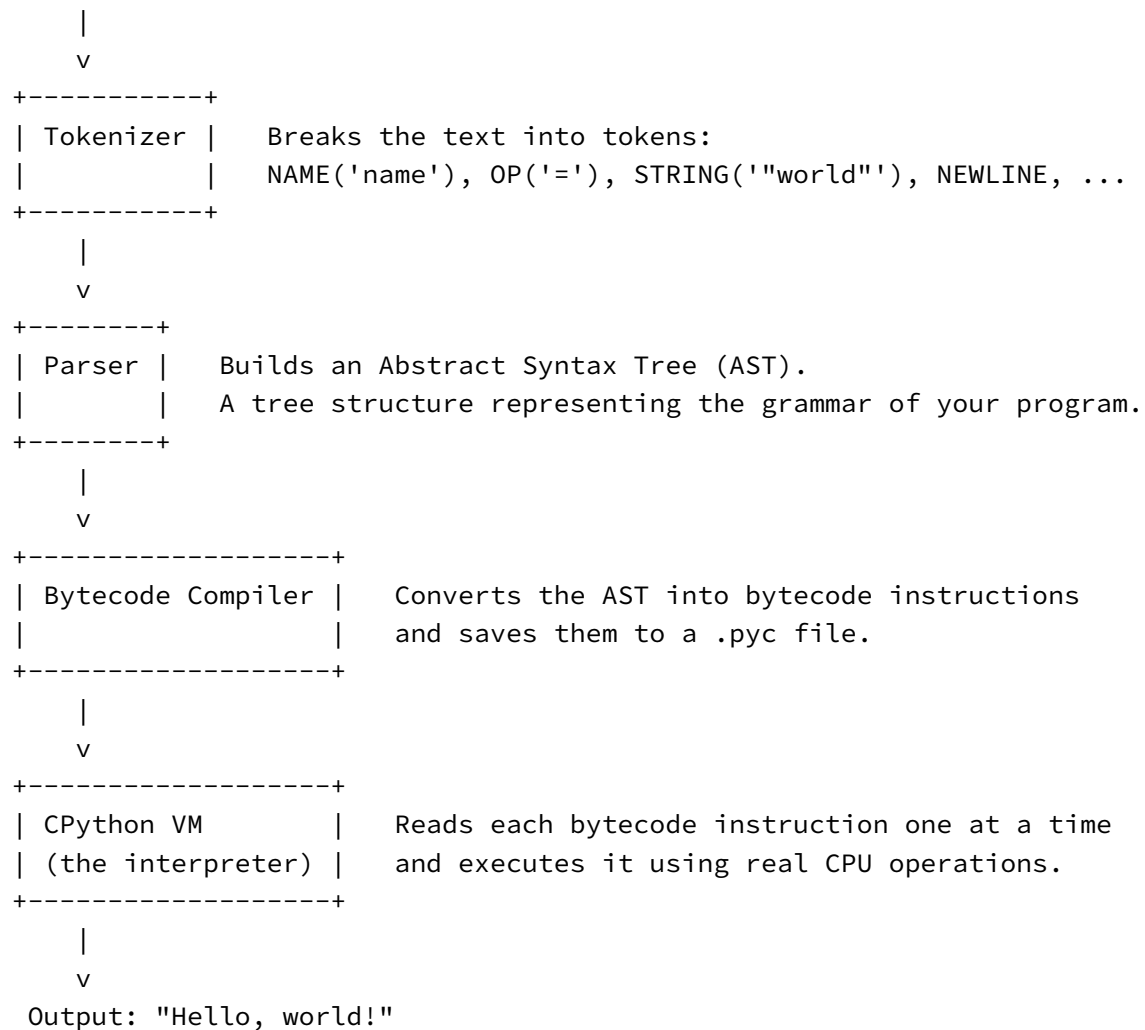
Let’s trace exactly what happens when you run a Python script.

```
# hello.py
name = "world"
print(f"Hello, {name}!")
```

```
$ python hello.py
Hello, world!
```

CPython (the standard Python interpreter) runs your code through five stages:

hello.py (plain text file)



The .pyc file contains instructions for a **virtual machine** – a software CPU that Python invented. These are NOT instructions your real CPU understands. The CPython VM translates each bytecode instruction into real CPU instructions as the program runs.

You can actually see the bytecode:

```

import dis

def greet(name):
    return f"Hello, {name}!"

dis.dis(greet)

```

Output:

```

2          0 RESUME                0

3          2 LOAD_CONST             1 ('Hello, ')
          4 LOAD_FAST               0 (name)

```

6	FORMAT_VALUE	0
8	BUILD_STRING	2
10	RETURN_VALUE	

LOAD_FAST, FORMAT_VALUE, BUILD_STRING – your CPU has never heard of these. The CPython VM reads each one and does the corresponding real work.

The consequence: There is always a middleman between your code and the CPU. Every Python operation passes through the interpreter loop. This is why Python is typically 10 to 100 times slower than C++ for CPU-bound work.

0.2.3 How C++ Runs Your Code

C++ uses a completely different model. Your source code is **translated directly into CPU instructions** before the program runs. There is no interpreter, no virtual machine, no middleman. When you run the program, your CPU executes your instructions directly.

This is called **ahead-of-time compilation**.

The translation is done by a chain of three programs:

```
hello.cpp (plain text, C++ source)
  |
  v
+-----+
| Preprocessor | Handles lines starting with #.
|              | Pastes #include files, substitutes #define macros.
|              | Pure text manipulation -- does NOT understand C++.
+-----+
  |
  | (preprocessed .cpp -- still text, but with all #includes pasted in)
  v
+-----+
| Compiler     | Parses C++, checks all types, optimizes, generates
|              | machine code for your CPU architecture.
+-----+
  |
  | (object file: .o -- real machine code, but references unresolved)
  v
+-----+
| Linker       | Combines all .o files plus library code into one
|              | self-contained executable.
+-----+
  |
  v
./hello (executable -- native machine code, runs directly on CPU)
```

0.2.3.1 Stage 1: The Preprocessor in Detail

The preprocessor is a dumb text tool. It knows only three things:

- `#include <file>` or `#include "file"` – paste the entire text of that file here
- `#define NAME text` – replace every occurrence of `NAME` with `text`
- `#ifdef` / `#ifndef` / `#else` / `#endif` – include or exclude blocks of text

It does not parse C++. It does not understand types. It just manipulates text.

When you write `#include <iostream>`, the preprocessor pastes thousands of lines into your file before the compiler sees anything. You can see how much:

```
$ g++ -E hello.cpp | wc -l
45231
```

One line expands to 45,231 lines. That is why large projects are slow to compile – every `.cpp` file re-expands every header it includes.

0.2.3.2 Stage 2: The Compiler in Detail

The compiler reads the preprocessed text and does the real work:

1. **Parsing:** builds an AST (same concept as Python's parser)
2. **Semantic analysis:** checks types, resolves names, catches errors
3. **Optimization** (with `-O2`): rewrites your code to run faster
4. **Code generation:** emits machine code for your target CPU

The output is an **object file** (`.o`). It contains real machine code, but it has gaps – references to functions defined in other files (like `std::cout`) that haven't been connected yet.

0.2.3.3 Stage 3: The Linker in Detail

The linker takes all object files and the standard library and combines them into one executable. It resolves every function call to an actual memory address. When `main.cpp` calls `std::cout`, the linker finds `cout`'s implementation in the standard library and wires them together.

0.2.3.4 The Core Difference

Python:

source --> bytecode (at first run) --> CPU (via interpreter, every run)

C++:

source --> machine code (once, at compile time) --> CPU (directly, every run)

After you compile once, running the program has zero translation overhead. The CPU just executes instructions.

0.2.4 Your First C++ Program

Create a file called `hello.cpp`:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!\n";
    return 0;
}
```

Compile and run:

```
$ g++ -std=c++23 -o hello hello.cpp
$ ./hello
Hello, world!
```

Now let's understand every token in this program.

0.2.4.1 `#include <iostream>`

A preprocessor directive. Pastes the `iostream` header (thousands of lines) into your file.

`iostream` declares `std::cout`, the standard output stream. Without it:

```
error: 'cout' is not a member of 'std'
  4 |     std::cout << "Hello, world!\n";
    |     ^~~~~~
note: 'std::cout' is defined in header '<iostream>';
     did you forget to '#include <iostream>'?
```

Angle brackets `<iostream>` mean “find this in the compiler’s system include path.” Double quotes `"my-file.h"` mean “look in my project directory first.”

0.2.4.2 `int main()`

The **entry point** of every C++ program. When the OS launches your executable, it calls `main()`. Everything begins here.

`int` is the **return type**. The function returns an integer to the OS as an exit code: `-0` = success (by convention) - Non-zero = some kind of failure

Unlike Python where top-level code runs directly, C++ requires exactly one function called `main` as the entry point. You can’t rename it or have two of them.

The `{}` braces delimit the function body, playing the same role as `:` and indentation in Python.

0.2.4.3 `std::cout << "Hello, world!\n";`

This is how you print to the terminal.

std is a **namespace** – a named container for related identifiers. The entire C++ standard library lives inside `std`. The `::` operator means “look inside.” So `std::cout` means “the `cout` that lives inside the `std` namespace.”

cout stands for “character output.” It is an object representing your terminal’s standard output.

`<<` is the **stream insertion operator**. It sends the value on the right into the stream on the left. You can chain it:

```
std::cout << "Hello" << ", " << "world" << "!\n";
```

Each `<<` sends one more thing into the stream.

`"Hello, world!\n"` is a string literal. The `\n` is a newline character. Unlike Python’s `print()`, which adds a newline automatically, C++ streams do not add newlines unless you include them.

`;` terminates the statement. Every statement ends with a semicolon. This is the single most common beginner error. Forgetting it produces an error on the *next* line because the compiler doesn’t notice until it sees an unexpected token.

0.2.4.4 `return 0;`

Returns the exit code 0 (success) to the OS. In `main` specifically, you can omit this and the compiler inserts it, but writing it is explicit and clear.

0.2.5 Compiling: What the Flags Mean

```
$ g++ -std=c++23 -Wall -Wextra -o hello hello.cpp
```

Flag	What it does
<code>g++</code>	The GNU C++ compiler. Also try <code>clang++</code> (often better error messages).
<code>-std=c++23</code>	Use the C++23 standard. Without this you get an older, less capable standard.
<code>-Wall</code>	Enable all common warnings. The W stands for Warning.
<code>-Wextra</code>	Enable even more warnings.
<code>-o hello</code>	Name the output executable <code>hello</code> . Default without <code>-o</code> is <code>a.out</code> .

Flag	What it does
hello.cpp	The source file to compile.

0.2.5.1 Two Build Modes You Will Use

```
# Development (use this while writing code):
$ g++ -std=c++23 -Wall -Wextra -g -fsanitize=address,undefined -o hello hello.cpp
#                                     |   |
#                                     |   +-- catches memory bugs and UB at runtime
#                                     +----- adds debug symbols for stack traces

# Release (use this for final builds):
$ g++ -std=c++23 -O2 -o hello hello.cpp
#                                     |
#                                     +-- enables optimization (2x-10x faster)
```

The sanitizers (`-fsanitize=address,undefined`) catch entire classes of bugs that would otherwise produce silent wrong answers or random crashes. Always use them during development.

0.2.6 What Is in the Compiled Binary?

The executable file is organized into sections:

`./hello` (ELF executable on Linux, Mach-O on macOS, PE on Windows)

Section	Contents
-----	-----
.text	Your compiled machine code instructions
.rodata	Read-only data: string literals like "Hello, world!\n"
.data	Initialized global variables
.bss	Uninitialized global variables (zeroed at program start)

You can disassemble the executable to see what your code became:

```
$ objdump -d -C hello | grep -A 15 "<main>"

0000000000001149 <main>:
   1149: push    rbp
   114a: mov     rbp, rsp
   114d: lea     rdi, [rip+0xeb0]    <- loads address of "Hello, world!\n"
   1154: call    1060 <puts@plt>    <- calls puts() (compiler optimized cout to this)
   1159: mov     eax, 0x0
   115e: pop     rbp
   115f: ret
```

Seven CPU instructions. No interpreter, no type checking, no reference counting. This is what runs when you execute `./hello`.

0.2.7 Multi-File Programs

Once programs grow beyond one file, you split them:

```
main.cpp  --compile-->  main.o  --+
                                     +--> linker --> program
greet.cpp --compile--> greet.o  --+
```

```
// greet.h -- declaration (the interface, what other files need to know)
#pragma once
#include <string>
void greet(const std::string& name);

// greet.cpp -- definition (the actual implementation)
#include <iostream>
#include "greet.h"
void greet(const std::string& name) {
    std::cout << "Hello, " << name << "!\n";
}

// main.cpp -- uses the function
#include "greet.h"
int main() {
    greet("world");
    return 0;
}
```

```
$ g++ -std=c++23 -o program main.cpp greet.cpp
```

Chapter 11 covers the full rationale for this split.

0.2.8 Reading Compiler Errors

The compiler is your most important tool. It catches errors before your program runs. Learning to read its output quickly is a core skill.

0.2.8.1 Missing Semicolon

```
int main() {
    std::cout << "Hello"    // missing semicolon here
    return 0;
}
```

```
hello.cpp:4:5: error: expected ';' before 'return'
   4 |     std::cout << "Hello"
     |                                     ^
     |                                     ;
   5 |     return 0;
```

The caret ^ marks where the problem was detected – the end of line 4. The message says “before ‘return’” because the compiler only noticed something was wrong when it hit `return`.

0.2.8.2 Missing #include

```
int main() {
    std::cout << "Hello\n";
}
```

```
hello.cpp:2:5: error: 'cout' is not a member of 'std'
   2 |     std::cout << "Hello\n";
     |     ^~~~~~
note: 'std::cout' is defined in header '<iostream>';
     did you forget to '#include <iostream>'?
```

Read every note: line. Modern compilers often tell you exactly what to do.

0.2.8.3 Undefined Reference (Linker Error)

```
// main.cpp
void greet(const std::string& name); // declared but never defined anywhere

int main() { greet("world"); }
```

```
/usr/bin/ld: main.o: undefined reference to `greet(std::string const&)'
collect2: error: ld returned 1 exit status
```

“Undefined reference” means the compiler accepted the code (it saw the declaration), but the linker couldn’t find the function’s body. Either you forgot to define it, or forgot to include its `.cpp` file in the build command.

0.2.8.4 Type Mismatch

```
int x = "hello";
```

error: invalid conversion from 'const char*' to 'int'

```
1 | int x = "hello";
  |           ^~~~~~
```

The static type system caught this before the program ran. Python would allow `x = 5; x = "hello"` – C++ refuses.

0.2.9 Common Mistakes in This Chapter

0.2.9.1 Mistake 1: Forgetting the semicolon

The bug: `std::cout << "Hello"` with no `;`

The symptom: Error message appears on the *next* line, not the line where you forgot it. The compiler doesn't notice until it encounters an unexpected token.

The fix: Every statement ends with `;`. After `return 0`. After every `cout`. After every variable declaration.

0.2.9.2 Mistake 2: Writing `cout` without `std::`

The bug:

```
cout << "Hello\n"; // 'cout' -- where is it?
```

The symptom:

error: 'cout' was not declared in this scope

The fix: Write `std::cout`. You can also add `using std::cout;` at the top of a function to avoid typing `std::` everytime. Do not use `using namespace std;` – it pulls in hundreds of names that may conflict with your own.

0.2.9.3 Mistake 3: Confusing `#include <...>` with `#include "..."`

The bug: Writing `#include "iostream"` instead of `#include <iostream>`

What happens: The preprocessor looks for `iostream` in the current directory first. It may find nothing and fail, or find the wrong file.

The fix: Use `<...>` for system and library headers. Use `"..."` for your own headers.

0.2.10 Exercises

Exercise 1.1 – Predict the output

What does this print? Work it out before running it.

```
#include <iostream>
int main() {
    std::cout << "Line 1\n";
    std::cout << "Line " << 2 << "\n";
    std::cout << "Line " << 1 + 2 << "\n";
    return 0;
}
```

Answer:

Line 1
Line 2
Line 3

1 + 2 is evaluated to 3 before being inserted into the stream.

Exercise 1.2 – Find all three bugs

This program has exactly three syntax errors. Find them all.

```
#include <iostream>
int main() {
    std::cout << "Hello, world!"
    std::cout << "How are you?"
    return 0
}
```

Answer: 1. Missing ; after "Hello, world!" 2. Missing ; after "How are you?" 3. Missing ; after return 0

Exercise 1.3 – Write from scratch

Write a program that prints this output exactly (copy the spacing):

```
=== My C++ Program ===
Author: Your Name
Version: 1.0
```

Answer:

```
#include <iostream>
int main() {
    std::cout << "=== My C++ Program ===\n";
    std::cout << "Author: Your Name\n";
    std::cout << "Version: 1.0\n";
    return 0;
}
```

Exercise 1.4 – Explain the difference

In your own words: what is the difference between compiling with `-O0` and `-O2`? Why would you use each?

Answer: `-O0` (no optimization) translates your code nearly literally into machine code. The machine code closely mirrors your source, making it easier to debug – variables are in the places you’d expect, and the program steps through code the way you wrote it. Use this during development.

`-O2` enables aggressive optimization: the compiler may inline functions, eliminate redundant computations, reorder instructions, and more. The program runs significantly faster. The machine code may look very different from your source. Use this for release builds.

Chapter 2: Variables, Types, and the Static Type System

0.2.11 What IS a Variable?

In Python, a variable is a **name tag** attached to an object. The object lives on the heap; the name is just a reference to it.

```
x = 42          # 'x' is a label pointing to an int object on the heap
y = x          # 'y' is another label pointing to the SAME object
x = "hello"    # 'x' now points to a different object; y is unchanged
print(y)       # 42
```

The object carries its own type:

Python memory:

Variable	Points to
x	[type: int refcount: 2 value: 42]
y	[same object]

After `x = "hello"`:

x	[type: str refcount: 1 value: "hello"]
y	[type: int refcount: 1 value: 42]

Every Python variable is a pointer. The object it points to carries its type, a reference count, and metadata.

In C++, a variable is a **named region of memory with a fixed type**. The variable IS the storage. There is no separate heap object (unless you explicitly create one), no reference count, no type tag at runtime.

```
int x = 42; // 'x' is 4 bytes of memory containing the bit pattern for 42
           // The type 'int' exists only in the compiler's mind
```

C++ memory:

Address	Variable	Bytes stored
0x7fff10	x (int)	[00][00][00][2A] <- that is 42 in hex

No type tag. No reference count. No metadata. Four bytes. That is it.

The type `int` tells the **compiler** (not the CPU, not the runtime): - Reserve 4 bytes - Treat those bytes as a signed two's complement integer - Allow operations like `+`, `-`, `*`, `/` - Reject operations like string concatenation

At runtime, there are only bytes. Types are a compile-time concept.

This is the most fundamental difference between Python and C++. Almost every other difference you will encounter flows from this one.

0.2.12 How Integers Are Stored: Binary and Two's Complement

To understand C++ types, you need to understand how integers are stored in memory. If you already know two's complement, skim this section.

A computer stores everything as bits: 0 or 1. Eight bits make one byte.

The number 42 in binary:

$$42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1$$

Bit positions (0 = rightmost):

7	6	5	4	3	2	1	0
0	0	1	0	1	0	1	0
		32		8		2	
0							

$$0 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 = 42$$

An `int` uses 4 bytes = 32 bits. So 42 stored as a 32-bit int:

Byte 3	Byte 2	Byte 1	Byte 0
00000000	00000000	00000000	00101010
= 42			

No type metadata anywhere. The CPU interprets these bytes according to which instruction operates on them.

0.2.12.1 How Negative Numbers Work: Two's Complement

For signed integers, the highest bit is the sign bit: 0 = positive, 1 = negative.

For a 32-bit signed int: - Maximum positive: 0111 1111 ... 1111 = 2,147,483,647 - Maximum negative: 1000 0000 ... 0000 = -2,147,483,648 - -1: 1111 1111 ... 1111 (all bits set)

To negate a number: flip all bits, then add 1.

42 in binary:	0000 0000 0000 0000 0000 0000 0010 1010
Flip all bits:	1111 1111 1111 1111 1111 1111 1101 0101
Add 1:	1111 1111 1111 1111 1111 1111 1101 0110 = -42

The critical implication: **signed integer overflow is undefined behavior in C++.**

```
int max = 2147483647;    // the largest possible int
int bad = max + 1;       // UNDEFINED BEHAVIOR
                        // The standard does not define what happens.
                        // In practice: wraps to -2147483648 on most systems.
                        // But the compiler is allowed to assume overflow
                        // never happens and optimize in ways that break you.
```

Python integers are arbitrary precision – they never overflow. C++ `int` has fixed size and hard limits. Always think about whether your values can exceed the range.

0.2.13 The Fundamental Types

0.2.13.1 bool

```
bool yes = true;
bool no  = false;

std::cout << yes << "\n"; // prints 1
std::cout << no  << "\n"; // prints 0
```

Stores true/false. Takes 1 byte (even though 1 bit would suffice – byte is the smallest addressable unit on most hardware). `true` is stored as 1, `false` as 0.

Any non-zero integer converts to `true`, zero converts to `false`:

```
bool b1 = 42; // true  (42 != 0)
bool b2 = 0;  // false
bool b3 = -1; // true  (-1 != 0)
```

0.2.13.2 char

```
char letter = 'A'; // single character, single quotes
char newline = '\n'; // escape sequences work like Python
char tab = '\t';
char null_c = '\0'; // null character (value 0)
```

One byte. Holds one ASCII character. The character is stored as its ASCII code: `'A'` = 65, `'Z'` = 90, `'0'` = 48, `'\n'` = 10.

You can do arithmetic on chars because they are just numbers:

```
char c = 'A';
std::cout << c + 1; // 66 (int arithmetic)
std::cout << static_cast<char>(c + 1); // 'B' (cast back to char)
std::cout << (char)('a' + 2); // 'c' (C-style cast, avoid)
```

0.2.13.3 Integer Types

Type	Typical size	Signed range	Notes
<code>short</code>	2 bytes	-32,768 to 32,767	Rarely used
<code>int</code>	4 bytes	-2.1B to 2.1B	Default integer type
<code>long</code>	4 or 8 bytes	platform-dependent	Avoid – use long long
<code>long long</code>	8 bytes	-9.2e18 to 9.2e18	Use when int overflows

To make a type unsigned (no negatives, larger maximum):

```
unsigned int u = 4000000000u; // 'u' suffix = unsigned literal
unsigned long long big = 18446744073709551615ull; // 'ull' suffix
```

An unsigned type with N bits holds 0 to $2^N - 1$. A signed type with N bits holds $-2^{(N-1)}$ to $2^{(N-1)} - 1$.

0.2.13.4 float and double

```
float f = 3.14f; // 32-bit floating point, 'f' suffix is required
double d = 3.14; // 64-bit floating point -- use this by default
```

double is what Python calls float. Use double unless you have a specific reason (memory, GPU/SIMD code).

Why prefer double? Precision: - float: approximately 7 significant decimal digits - double: approximately 15 significant decimal digits

```
float f = 1.23456789f;
double d = 1.23456789;
std::cout << std::setprecision(10) << f << "\n"; // 1.234567881 (wrong at 7th digit)
std::cout << std::setprecision(10) << d << "\n"; // 1.23456789 (correct)
```

Floating point is not exact – same in Python and C++:

```
# Python
print(0.1 + 0.2) # 0.30000000000000004
```

```
// C++
#include <iomanip>
double x = 0.1 + 0.2;
std::cout << x << "\n"; // 0.3 (default rounds nicely)
std::cout << std::setprecision(17) << x << "\n"; // 0.30000000000000004
std::cout << std::setprecision(17) << 0.3 << "\n"; // 0.29999999999999999
```

0.1, 0.2, and 0.3 cannot be represented exactly in binary floating point – the same way $1/3$ cannot be represented exactly in decimal. This is not a C++ bug; it is IEEE 754 floating point. The implication: **never compare floating-point numbers with ==.**

0.2.13.5 std::string

```
#include <string>
std::string name = "Alice";
std::string greeting = "Hello, " + name + "!"; // concatenation with +
std::cout << greeting.size() << "\n"; // 13
```

Not a primitive type. `std::string` is a class from the standard library that manages heap-allocated character data. We cover it in depth in Chapter 10.

0.2.14 Fixed-Width Integer Types

The sizes of `short`, `int`, `long` depend on the platform and compiler. For code that must work correctly regardless of platform – networking, file formats, hardware interfaces – use exact-size types from `<cstdint>`:

```
#include <cstdint>

int8_t  a = -128;           // exactly 8 bits, signed (-128 to 127)
uint8_t b = 255;           // exactly 8 bits, unsigned (0 to 255)
int16_t c = 32767;         // exactly 16 bits, signed
uint16_t d = 65535;        // exactly 16 bits, unsigned
int32_t e = -1;            // exactly 32 bits, signed
uint32_t f = 4294967295u;   // exactly 32 bits, unsigned
int64_t g = -1LL;          // exactly 64 bits, signed
uint64_t h = 0xFFFFFFFFFFFFFFFFull; // exactly 64 bits, unsigned, all bits set
```

Use these whenever the exact bit width matters. Use `int` when it does not.

0.2.15 Declaring and Initializing Variables

0.2.15.1 Never Leave a Variable Uninitialized

This is the most important rule in this chapter:

```
int sum;                                // BAD: sum contains whatever bytes were
for (int i = 0; i < 10; ++i)           // in that memory location before.
    sum += i;                          // Adding to garbage = garbage.
std::cout << sum << "\n";              // Prints something unpredictable.

int sum{0};                             // GOOD: explicitly initialized to zero.
for (int i = 0; i < 10; ++i)
    sum += i;
std::cout << sum << "\n";              // 45, always.
```

On some runs, the garbage value might happen to be 0, making the bug invisible. On other runs, it will be -858993460 or similar. Undefined behavior is unpredictable.

0.2.15.2 The Four Initialization Forms

C++ has four ways to initialize a variable. They have subtle differences.

Form 1: Default initialization (leaves value unset – dangerous)

```
int x;           // x has garbage value
double d;        // d has garbage value
```

Do not do this for primitive types.

Form 2: Copy initialization (C-style, from C)

```
int x = 5;
double d = 3.14;
std::string s = "hello";
```

Looks like assignment but is initialization (happens at construction, not after). Fine for simple cases.

Form 3: Direct initialization

```
int x(5);
double d(3.14);
std::string s("hello");
```

More verbose for primitives but is the natural form for class objects with constructors.

Form 4: Brace (uniform) initialization – prefer this

```
int x{5};
double d{3.14};
std::string s{"hello"};
```

Why prefer brace initialization? It prevents **narrowing conversions** – type truncations that silently discard data:

```
// With = initialization, narrowing is SILENT:
int a = 3.7;           // truncates to 3. No error. Potential bug hiding.
int b = 300000;        // fits in int, fine
int c = 300000LL;      // long long to int, may truncate on 16-bit systems, no warning

// With {} initialization, narrowing is a COMPILE ERROR:
int a{3.7};           // error: narrowing conversion of '3.7e+0' from 'double' to 'int'
int b{300000};        // fine
int c{300000LL};      // fine (value fits in int)
```

The compiler tells you about the truncation at compile time instead of letting it silently corrupt your data at runtime.

You can value-initialize (set to zero/false/null) with empty braces:

```
int    n{};    // 0
double d{};    // 0.0
bool   b{};    // false
char   c{};    // '\0'
```

0.2.16 Memory Layout of Variables

```
bool   a{true};    // 1 byte
char   b{'X'};     // 1 byte
int    c{42};      // 4 bytes
double d{3.14};    // 8 bytes
```

Stack memory (addresses are illustrative):

Addr	Var	Size	Raw bytes (hex)	
0x1000	a (bool)	1 byte	[01]	
0x1001	b (char)	1 byte	[58]	'X' = ASCII 88 = 0x58
0x1004	c (int)	4 bytes	[00][00][00][2A]	42 = 0x2A
0x1008	d (double)	8 bytes	[40][09][1E][B8][51][EB][85][1F]	

(The actual layout may differ due to alignment, but the sizes are as shown.)

There is no type information stored at runtime. The CPU knows to treat `c` as a signed integer because the compiled instructions operating on it are integer instructions. The type system is entirely a compile-time construct.

0.2.17 auto – Type Deduction

When the type is obvious from the right-hand side, `auto` lets the compiler deduce it:

```
auto x = 42;           // int    (integer literals default to int)
auto y = 42L;          // long   (L suffix)
auto z = 42LL;         // long long (LL suffix)
auto d = 3.14;         // double (decimal literals default to double)
auto f = 3.14f;        // float  (f suffix)
auto b = true;         // bool
auto s = std::string{"hi"}; // std::string
```

`auto` does NOT make C++ dynamically typed. The type is deduced once at compile time and then fixed permanently:

```

auto x = 42;           // x is int, fixed forever
x = 3.14;              // fine: double 3.14 truncates to int 3 (with = init)
x = "hello";           // COMPILE ERROR: cannot assign const char* to int

```

`auto` strips top-level `const` and references from the deduced type. Add them explicitly if you need them:

```

int n = 5;
auto a = n;           // int (copy)
auto& b = n;          // int& (reference to n)
const auto& c = n;    // const int& (const reference, no copy, no modify)

```

Where `auto` really shines is avoiding verbose type names:

```

// Without auto:
std::vector<std::pair<std::string, int>>::iterator it = scores.begin();

// With auto (same type, more readable):
auto it = scores.begin();

```

0.2.18 `const` and `constexpr`

0.2.18.1 `const` – Constant After Initialization

```

const int MAX = 8;
MAX = 10;           // COMPILE ERROR: assignment of read-only variable

const int n = get_input(); // value known only at runtime -- still valid const

```

Use `const` for values that are initialized once and never change. The compiler enforces this – it's not just a convention.

0.2.18.2 `constexpr` – Must Be Computed at Compile Time

```

constexpr int BOARD = 8;
constexpr double PI = 3.14159265358979;

int grid[BOARD][BOARD]; // OK -- array size must be a compile-time constant

constexpr int square(int x) { return x * x; }
constexpr int area = square(BOARD); // 64, computed at compile time, zero runtime cost

```

`constexpr` is stronger than `const`. The value must be computable without running the program. If it isn't, you get a compile error – which is what you want.

Rule: use `constexpr` for mathematical constants, array sizes, and configuration values. Use `const` for values set at runtime that shouldn't change afterward.

0.2.19 Type Conversion

0.2.19.1 Safe Widening (Automatic)

```
int    i = 42;
double d = i;    // int to double: no data loss, happens automatically
long long l = i; // int to long long: always safe
```

0.2.19.2 Narrowing (Dangerous Without Braces)

```
double d = 3.99;
int    i = d;    // silently truncates to 3 (with = initialization)
int    j{d};     // COMPILE ERROR: narrowing (with {} initialization)
```

0.2.19.3 Explicit Conversion with `static_cast`

When you intend to convert, be explicit about it:

```
double d = 3.99;
int i = static_cast<int>(d);    // 3, explicit truncation, intent is documented

int a = 5, b = 2;
// Bug: division happens as int first, then converts to double
double wrong = a / b;           // 2.0

// Fix: cast one operand to double before dividing
double correct = static_cast<double>(a) / b; // 2.5
```

`static_cast<NewType>(expr)` is the C++ way to convert. It is checked at compile time. Do not use C-style casts like `(int)x` – they bypass some safety checks and are harder to search for in code.

0.2.20 Checking Sizes with `sizeof`

```
#include <iostream>
int main() {
    std::cout << "bool:      " << sizeof(bool) << " byte(s)\n"; // 1
    std::cout << "char:      " << sizeof(char) << " byte(s)\n"; // 1
    std::cout << "int:       " << sizeof(int) << " byte(s)\n"; // 4
    std::cout << "long long: " << sizeof(long long) << " byte(s)\n"; // 8
    std::cout << "float:     " << sizeof(float) << " byte(s)\n"; // 4
    std::cout << "double:    " << sizeof(double) << " byte(s)\n"; // 8
    return 0;
}
```

`sizeof` is evaluated at compile time – zero runtime overhead. It works on both types (`sizeof(int)`) and variables (`sizeof(x)`).

0.2.21 Common Mistakes in This Chapter

0.2.21.1 Mistake 1: Uninitialized Variable

The bug:

```
int total;
for (int i = 1; i <= 5; ++i)
    total += i; // total starts with garbage, not 0
std::cout << total;
```

The symptom: Wrong or unpredictable output. Sometimes appears correct (when garbage happens to be 0).

The fix: `int total{0};`

How to detect: Compile with `-Wall` (warns about “total may be used uninitialized”). Run with `-fsanitize=undefined` for runtime detection.

0.2.21.2 Mistake 2: Signed Integer Overflow

The bug:

```
int seconds_per_year = 365 * 24 * 60 * 60; // 31,536,000 -- fits in int
int seconds_per_century = seconds_per_year * 100; // 3,153,600,000 -- OVERFLOW!
// int max is ~2.1 billion; 3.15 billion overflows
```

The symptom: Wrong answer (often a negative number). No compile error.

The fix: `long long seconds_per_century = (long long)seconds_per_year * 100;`

How to detect: `-fsanitize=undefined` reports “signed integer overflow.”

0.2.21.3 Mistake 3: Integer Division When Expecting Decimal

The bug:

```
double average = (100 + 75 + 90) / 3; // 88 -- not 88.333...
```

All literals are `int`. The division is integer division before the result is stored in `double`.

The fix: `double average = (100 + 75 + 90) / 3.0;` or use `static_cast<double>`.

0.2.21.4 Mistake 4: Comparing Floats with `==`

The bug:

```
double x = 0.1 + 0.2;
if (x == 0.3) {
    std::cout << "equal\n"; // never prints
}
```

The fix:

```
#include <cmath>
if (std::abs(x - 0.3) < 1e-9) {
    std::cout << "equal\n"; // works
}
```

0.2.22 Exercises

Exercise 2.1 – Type sizes

Without looking anything up, predict the output of this program on a 64-bit Linux system, then verify:

```
#include <iostream>
int main() {
    std::cout << sizeof(bool) << "\n";
    std::cout << sizeof(char) << "\n";
    std::cout << sizeof(short) << "\n";
    std::cout << sizeof(int) << "\n";
    std::cout << sizeof(long) << "\n";
    std::cout << sizeof(long long) << "\n";
    std::cout << sizeof(float) << "\n";
    std::cout << sizeof(double) << "\n";
}
```

Answer: 1 1 2 4 8 8 4 8 on 64-bit Linux. Note that `long` is 8 bytes on 64-bit Linux but 4 bytes on 64-bit Windows – another reason to use `long long` when you need 64 bits.

Exercise 2.2 – Brace initialization rejection

Which of these lines would brace initialization reject with a compile error?

```
int a{3.0}; // (a)
int b{3}; // (b)
int c{3LL}; // (c) LL = long long literal
double d{3}; // (d)
float e{3.14}; // (e)
```

Answer: - (a): double -> int narrows (loses the decimal). **ERROR.** - (b): int -> int, exact. Fine. - (c): long long -> int may narrow if value doesn't fit. **ERROR** (even though 3 fits, the types differ). - (d): int -> double widens (no data loss). Fine. - (e): double -> float narrows (loses precision). **ERROR.**

Exercise 2.3 – Fix the division

Fix this code so it prints 2.5 instead of 2:

```
#include <iostream>
int main() {
    int a = 5, b = 2;
    std::cout << a / b << "\n";
}
```

Answer:

```
std::cout << static_cast<double>(a) / b << "\n";
// or:
std::cout << a / static_cast<double>(b) << "\n";
// or:
std::cout << a / 2.0 << "\n";
```

Exercise 2.4 – Max value calculation

What is the maximum value of `uint32_t`? Show the calculation. Do the same for `int32_t`.

Answer: - `uint32_t`: 32 bits, unsigned. Range = 0 to $2^{32} - 1 = \mathbf{4,294,967,295}$. - `int32_t`: 32 bits, signed. Range = -2^{31} to $2^{31} - 1 = \mathbf{-2,147,483,648}$ to $\mathbf{2,147,483,647}$. One bit is used for the sign.

Chapter 3: Operators and Expressions

0.2.23 Arithmetic Operators

Most arithmetic operators work the same as Python. Division is the exception that trips everyone up.

```
int a = 17, b = 5;

std::cout << a + b << "\n"; // 22 -- addition
std::cout << a - b << "\n"; // 12 -- subtraction
std::cout << a * b << "\n"; // 85 -- multiplication
std::cout << a / b << "\n"; // 3 -- WATCH OUT: integer division
std::cout << a % b << "\n"; // 2 -- remainder (modulo)
```

The division rule: **when both operands are integers, / discards the remainder.**

```
# Python: / always returns a float
print(17 / 5) # 3.4
print(17 // 5) # 3 (floor division, rounds toward negative infinity)
print(-17 // 5) # -4 (floor: -3.4 rounds down to -4)
```

```
// C++: / on integers truncates toward zero (not floor)
std::cout << 17 / 5 << "\n"; // 3 (truncated toward zero)
std::cout << -17 / 5 << "\n"; // -3 (truncated toward zero, NOT -4)
std::cout << 17.0 / 5 << "\n"; // 3.4 (float division: 17.0 is a double)
```

The key distinction: C++ truncates toward zero. Python's `//` floors toward negative infinity. For positive numbers they agree. For negative numbers they differ.

0.2.23.1 The Classic Division Trap

```
int numerator = 7;
int denominator = 2;

double result = numerator / denominator; // 3.0, NOT 3.5
```

What happened? The division `numerator / denominator` is evaluated first – both are `int`, so integer division gives 3. Then 3 is converted to 3.0 for the `double` assignment.

The fix: make at least one operand a `double` before the division:

```
double result = static_cast<double>(numerator) / denominator; // 3.5
// or:
double result = numerator / static_cast<double>(denominator); // 3.5
// or (if one is a literal):
double result = numerator / 2.0; // 3.5
```

0.2.23.2 Modulo With Negative Numbers

```
std::cout << 7 % 3 << "\n"; // 1 (7 = 2*3 + 1)
std::cout << -7 % 3 << "\n"; // -1 (sign follows dividend in C++)
std::cout << 7 % -3 << "\n"; // 1 (sign follows dividend)
```

Python's `%` always returns non-negative when the divisor is positive (e.g., `-7 % 3 = 2` in Python). C++'s `%` sign follows the dividend.

This matters for “wrap around” patterns. To get Python-like non-negative modulo:

```
int python_mod(int a, int b) {
    return ((a % b) + b) % b;
}
// python_mod(-7, 3) = ((-1) + 3) % 3 = 2 % 3 = 2 -- same as Python
```

0.2.24 Increment and Decrement Operators

C++ has `++` (add 1) and `--` (subtract 1) as standalone operators. Python does not.

```
int x = 5;
++x; // pre-increment: x becomes 6
x++; // post-increment: x becomes 7
--x; // pre-decrement: x becomes 6
x--; // post-decrement: x becomes 5
```

Used as standalone statements, `++x` and `x++` are identical in effect. The difference appears when used inside a larger expression:

```
int x = 5;
int a = ++x; // pre: x becomes 6 FIRST, then a = 6 (new value)
int b = x++; // post: b = 6 (old value), then x becomes 7
```

Pre-increment (++x):

Step 1: increment x (x = 7)

Step 2: expression evaluates to new x (7)

Post-increment (x++):

Step 1: save current x (6)

Step 2: increment x (x = 7)

Step 3: expression evaluates to saved old x (6)

Always prefer ++x (pre-increment) as a habit. It is never slower than x++, and for iterator objects (Chapter 28) it can be significantly faster because post-increment must save a copy of the old value.

0.2.25 Compound Assignment

These modify a variable in place:

```
int x = 20;
x += 5;    // x = 25    (same as x = x + 5)
x -= 3;    // x = 22
x *= 2;    // x = 44
x /= 4;    // x = 11    (integer division if x is int)
x %= 3;    // x = 2      (remainder)
x <<= 2;   // x = 8      (left shift by 2: same as x * 4)
x >>= 1;   // x = 4      (right shift by 1: same as x / 2 for non-negative)
```

0.2.26 Comparison Operators

All return bool:

```
int a = 5, b = 10;
bool r1 = (a == b);    // false -- equal
bool r2 = (a != b);    // true  -- not equal
bool r3 = (a < b);     // true  -- less than
bool r4 = (a > b);     // false -- greater than
bool r5 = (a <= b);    // true  -- less than or equal
bool r6 = (a >= b);    // false -- greater than or equal
```

0.2.26.1 The Floating-Point Comparison Problem

```
double x = 0.1 + 0.2;
double y = 0.3;
std::cout << (x == y) << "\n";    // 0 (false!)
```

0.1 cannot be represented exactly in binary. Neither can 0.2 or 0.3. The sum 0.1 + 0.2 comes out as 0.30000000000000004... Stored 0.3 is 0.29999999999999998... They are not equal, even though mathematically they should be.

This is not a C++ problem – the same thing happens in Python and every other language that uses IEEE 754 floating point (which is all of them).

Rule: Never compare floating-point numbers with ==. Use a tolerance.

```
#include <cmath>    // for std::abs

// Returns true if a and b are within tolerance of each other
bool nearly_equal(double a, double b, double tolerance = 1e-9) {
    return std::abs(a - b) < tolerance;
}

if (nearly_equal(0.1 + 0.2, 0.3)) {
    std::cout << "equal (within tolerance)\n"; // this prints
}
```

Choose tolerance based on the scale of your values: - For values around 1.0: 1e-9 is reasonable - For values around 1,000,000: 1e-3 may be needed - For values that went through many operations: use larger tolerances

0.2.27 Logical Operators

```
bool x = true, y = false;

bool and_result = x && y;    // false (both must be true) -- Python: x and y
bool or_result  = x || y;    // true  (either must be true) -- Python: x or y
bool not_result = !x;        // false (flip)                -- Python: not x
```

0.2.27.1 Short-Circuit Evaluation

&& stops evaluating as soon as it finds a false. || stops as soon as it finds a true. This is identical to Python and is critically important for safety:

```

int* p = nullptr;

// Safe: if p is null, the right side is NEVER evaluated
if (p != nullptr && *p > 0) {
    std::cout << "positive pointee\n";
}
// Without short-circuit: *p when p is null = crash

// Common optimization: cheap check first, expensive check second
if (is_in_cache(key) || compute_and_cache(key)) {
    // compute_and_cache only called when not in cache
}

```

Short-circuit evaluation is relied upon for correctness, not just optimization.

0.2.28 Bitwise Operators

These operate on individual bits within integers. They appear constantly in systems programming, graphics, embedded code, and game engines.

Op	Name	Meaning
&	AND	Output bit is 1 only if BOTH input bits are 1
\	OR	Output bit is 1 if EITHER input bit is 1
^	XOR	Output bit is 1 if the two input bits are DIFFERENT
~	NOT	Flips all bits
<<	Left shift	Shifts bits left, filling with 0 on the right
>>	Right shift	Shifts bits right (fills with 0 for unsigned, sign bit for signed)

Example: a = 5 = 0101 in binary
b = 3 = 0011 in binary

```

a & b:  0101
        0011
        ----
        0001   = 1   (only bit 0 is set in both)

```

```

a | b:  0101
        0011
        ----
        0111   = 7   (any bit set in either)

```

```
a ^ b:  0101
        0011
        ----
        0110  = 6  (bits that differ)
```

```
~a:      ~0101 = 1111...11111010 = -6 (for 32-bit signed int)
```

Left shift by 1 = multiply by 2. Left shift by n = multiply by 2^n:

```
1 << 0 = 1
1 << 1 = 2
1 << 2 = 4
1 << 3 = 8
1 << 4 = 16
1 << 10 = 1024
```

0.2.28.1 Practical Use: Boolean Flags Packed in an Integer

Instead of a separate `bool` for each property, pack them into one integer – each bit represents one flag:

```
const uint32_t VISIBLE    = 1u << 0;  // bit 0: 00000001
const uint32_t SOLID     = 1u << 1;  // bit 1: 00000010
const uint32_t ACTIVE    = 1u << 2;  // bit 2: 00000100
const uint32_t HAS_HEALTH = 1u << 3;  // bit 3: 00001000

uint32_t entity_flags = 0;

// Set a flag (turn bit ON):
entity_flags |= VISIBLE;    // flags = 00000001
entity_flags |= ACTIVE;    // flags = 00000101

// Test a flag (check if bit is set):
if (entity_flags & VISIBLE) { // nonzero = true
    render(entity);
}
if (entity_flags & ACTIVE) {
    update(entity);
}

// Clear a flag (turn bit OFF):
entity_flags &= ~ACTIVE;    // ~ACTIVE = 11111011; AND clears bit 2
                           // flags = 00000001

// Toggle a flag (flip):
entity_flags ^= SOLID;     // XOR flips bit 1
```

This pattern is everywhere: OpenGL uses `GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT`, Vulkan uses `VkImageUsageFlags`, Linux uses `O_RDONLY | O_CREAT` for file open flags.

0.2.29 Operator Precedence

When operators are mixed, precedence determines the order of evaluation:

Highest priority (evaluated first)

() [] -> .	grouping, subscript, member access
! ~ ++x --x (unary)	unary operators (right-to-left)
* / %	multiplicative
+ -	additive
<< >>	bitwise shift
< <= > >=	relational comparison
== !=	equality comparison
&	bitwise AND
^	bitwise XOR
	bitwise OR
&&	logical AND
	logical OR
?:	ternary (right-to-left)
= += -= ...	assignment (right-to-left)

Lowest priority (evaluated last)

Common traps:

```
// Trap 1: bitwise AND vs equality
if (flags & MASK == 1) { }    // parsed as: flags & (MASK == 1)  -- WRONG
if ((flags & MASK) == 1) { }  // correct

// Trap 2: shift and arithmetic
int x = 2 + 3 << 1;    // parsed as: 2 + (3 << 1) = 2 + 6 = 8
int y = (2 + 3) << 1;  // = 5 << 1 = 10

// Rule: when in doubt, parenthesize
```

0.2.30 The Ternary Operator

```
# Python
label = "even" if n % 2 == 0 else "odd"
```

```
// C++
std::string label = (n % 2 == 0) ? "even" : "odd";
//                condition   true    false
```

The ternary is an *expression* that produces a value. Good for simple one-liners. Do not nest them – `a ? b ? c : d : e` is unreadable.

0.2.31 Common Mistakes in This Chapter

0.2.31.1 Mistake 1: Assignment Instead of Comparison

The bug:

```
if (x = 5) { ... } // assigns 5 to x, condition is always true (5 != 0)
```

The symptom: The `if` branch always runs; the `else` never runs; `x` silently changed. **The fix:** `if (x == 5)`. Compile with `-Wall` to get a warning about this. **Defensive coding:** Some people write `if (5 == x)` (“Yoda conditions”). Then a typo `if (5 = x)` is a compile error (can’t assign to a literal).

0.2.31.2 Mistake 2: Integer Division Surprise

The bug:

```
double bmi = weight / (height * height); // all ints -- integer division!
```

The fix: Cast one operand: `static_cast<double>(weight) / (height * height)`

0.2.31.3 Mistake 3: == on Floats

The bug: `if (result == expected_value)` where both are doubles **The fix:** `if (std::abs(result - expected_value) < 1e-9)`

0.2.31.4 Mistake 4: Bitwise vs Logical Operators

The bug:

```
bool a = is_ready();
bool b = can_proceed();
if (a & b) { ... } // bitwise AND -- works only if a and b are 0 or 1
// breaks for general integers
if (a && b) { ... } // logical AND -- correct for boolean conditions
```

The fix: Use `&&` and `||` for boolean logic. Use `&` and `|` for bit manipulation.

0.2.32 Exercises

Exercise 3.1 – Predict the output

```
int a = 17, b = 5;
std::cout << a / b << "\n";
std::cout << a % b << "\n";
std::cout << -a / b << "\n";
std::cout << -a % b << "\n";
```

Answer: 3, 2, -3, -2. C++ truncates toward zero; the sign of % follows the dividend.

Exercise 3.2 – Bit flag operations

Starting from `uint8_t flags = 0b00000000`: 1. Set bit 2 (value 4) 2. Set bit 5 (value 32) 3. Check if bit 2 is set (should be true) 4. Clear bit 2

Write the four operations and show the final value of `flags`.

Answer:

```
uint8_t flags = 0b00000000;

flags |= (1u << 2);           // 1. set bit 2: flags = 0b00000100
flags |= (1u << 5);           // 2. set bit 5: flags = 0b00100100

bool bit2_set = (flags & (1u << 2)) != 0; // 3. check bit 2: true

flags &= ~(1u << 2);           // 4. clear bit 2: flags = 0b00100000
// ~(1u << 2) = ~0b00000100 = 0b11111011
// 0b00100100 & 0b11111011 = 0b00100000 = 32
```

Exercise 3.3 – Floating-point tolerance

Write a function `are_equal(double a, double b)` that returns true when `a` and `b` are within 0.0001 of each other. Test it with `0.1 + 0.2` vs `0.3`.

Answer:

```
#include <cmath>
bool are_equal(double a, double b) {
    return std::abs(a - b) < 0.0001;
}
// are_equal(0.1 + 0.2, 0.3) returns true
// are_equal(0.1 + 0.2, 0.4) returns false
```

0.2.33 if/else if/else

```
# Python
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

```
// C++
int score = 85;
std::string grade;

if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else {
    grade = "C";
}
```

Three syntax differences: 1. The condition must be in **parentheses**: `if (score >= 90)` not `if score >= 90` 2. The body uses **curly braces** `{}` instead of `:` and indentation 3. **else if** (two words, with a space) not `elif`

0.2.33.1 Always Use Curly Braces – A Real Cautionary Tale

Braces are technically optional when the body is a single statement:

```
if (error)
    std::cout << "Error occurred\n";    // technically fine
```

But do not do this. The Apple “goto fail” bug (2014) used this pattern and allowed attackers to bypass SSL certificate verification on millions of Apple devices. The vulnerable code looked like:

```
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
goto fail;    // This ALWAYS executes -- attacker bypasses verification
```

The second `goto fail` is not inside the `if`. The indentation lies. Always use braces.

0.2.33.2 if with Initializer (C++17)

Declare a variable scoped only to the `if/else` block:

```
// C++ before 2017:
int value = compute();
if (value > 0) {
    use(value);
}
// 'value' still exists here (wasted scope, potential confusion)

// C++17 and later:
if (int value = compute(); value > 0) {
    // ^-- initializer      ^-- condition
    use(value);
} else {
    handle_error(value);
}
// 'value' does not exist here -- scope is clean
```

Use this when the variable is only meaningful within the `if/else`.

0.2.34 switch

`switch` tests one integer or enum value against a list of constants:

```
int day = 3;

switch (day) {
    case 1:
        std::cout << "Monday\n";
        break;
    case 2:
        std::cout << "Tuesday\n";
        break;
    case 3:
        std::cout << "Wednesday\n";
        break;
    case 6:
    case 7:
        std::cout << "Weekend\n";    // cases 6 and 7 share the same body
        break;
    default:
        std::cout << "Weekday\n";
        break;
}
```

0.2.34.1 Fallthrough: The break Requirement

Without `break`, execution falls through into the next case:

```
int x = 1;
switch (x) {
    case 1:
        std::cout << "one\n";
        // no break!
    case 2:
        std::cout << "two\n";
        break;
    case 3:
        std::cout << "three\n";
        break;
}
```

Output when `x == 1`: one then two. The control falls through from case 1 into case 2.

This is almost always a bug. Always add `break`. If fallthrough is intentional, use `[[fallthrough]]` (C++17) to document it and silence the warning:

```
case 'A':
    [[fallthrough]]; // intentional: A and a are treated the same
case 'a':
    process_letter_a();
    break;
```

`switch` only works on integral types (`int`, `char`, `enum`). It does not work on strings or floating-point numbers.

0.2.35 while Loop

Checks the condition before each iteration. If the condition is false initially, the body never runs.

```
# Python
x = 0
while x < 5:
    print(x)
    x += 1
```

```
// C++
int x = 0;
while (x < 5) {
    std::cout << x << "\n";
    ++x;
}
// prints 0, 1, 2, 3, 4
```

0.2.36 do-while Loop

The body runs at least once. The condition is checked after the body.

```
int x = 100;
do {
    std::cout << x << "\n"; // prints 100 even though 100 >= 5
    ++x;
} while (x < 5);
// body ran once; condition was false; loop ends
```

When to use it: Input validation – you always want to ask once, then check.

```
#include <iostream>

int main() {
    int age;
    do {
        std::cout << "Enter your age (1-120): ";
        std::cin >> age;
    } while (age < 1 || age > 120);
    // Guaranteed: asked at least once; keeps asking until valid input

    std::cout << "Age accepted: " << age << "\n";
    return 0;
}
```

Python has no do-while. The equivalent Python idiom:

```
while True:
    age = int(input("Enter your age (1-120): "))
    if 1 <= age <= 120:
        break
```

0.2.37 for Loop: C-Style

```
for (initializer; condition; increment) {
    body;
}
```

```
for (int i = 0; i < 10; ++i) {
    std::cout << i << " "; // 0 1 2 3 4 5 6 7 8 9
}
// 'i' does not exist after the loop -- scoped to the for
```

Execution order:

```

for (int i = 0; i < 10; ++i)

    int i = 0      <-- runs ONCE before anything
      |
      v
    [check: i < 10?]  ----NO----> exit loop
      |
      YES
      |
      v
    [body: cout << i]
      |
      v
    [++i]
      |
      +-----> [check: i < 10?] (repeat)

```

Key point: the variable declared in the initializer (`int i`) exists only inside the `for` loop. This is better than Python, where loop variables leak out:

```

# Python: loop variable persists after loop
for i in range(5):
    pass
print(i)  # 4 -- still accessible

```

```

// C++: loop variable is gone after loop
for (int i = 0; i < 5; ++i) { }
std::cout << i;  // COMPILE ERROR: 'i' was not declared in this scope

```

Useful patterns:

```

// Count down:
for (int i = 10; i >= 0; --i) {
    std::cout << i << " ";  // 10 9 8 7 6 5 4 3 2 1 0
}

// Step by 3 (like range(0, 30, 3)):
for (int i = 0; i < 30; i += 3) {
    std::cout << i << " ";  // 0 3 6 9 12 15 18 21 24 27
}

// Multiple variables:
for (int i = 0, j = 10; i < j; ++i, --j) {
    std::cout << i << " " << j << "\n";
}
// 0 10, 1 9, 2 8, 3 7, 4 6

```

0.2.38 Range-Based for Loop (C++11)

The C++ equivalent of Python's `for x in collection`:

```
numbers = [10, 20, 30, 40, 50]
for n in numbers:
    print(n)
```

```
#include <vector>
std::vector<int> numbers = {10, 20, 30, 40, 50};
for (int n : numbers) {
    std::cout << n << "\n";
}
```

0.2.38.1 The Reference vs Copy Decision

This is the most important thing to understand about range-based for in C++.

By value (copy): Each iteration creates a copy. Modifying the loop variable does NOT change the container.

```
std::vector<int> v = {1, 2, 3};
for (int n : v) {
    n *= 10;    // modifies the copy, not the element in v
}
// v is still {1, 2, 3}
```

By reference (&): The loop variable IS the element. Modifying it DOES change the container.

```
for (int& n : v) {
    n *= 10;    // modifies the actual element in v
}
// v is now {10, 20, 30}
```

By const reference (const&): Read-only access to the element, no copy made. Use this for large objects you only need to read.

```
for (const int& n : v) {
    std::cout << n << "\n";    // read-only, no copy
    // n = 0;    // COMPILE ERROR: n is const
}
```

When to use which:

Type is small (int, double, bool, char, pointer):

```
for (T x : v)            -- copy is cheap, fine
for (const T& x : v)     -- also fine
```

Type is large (string, struct, vector, custom class):

```
for (const T& x : v)    -- ALWAYS prefer: no copy, no modify
for (T& x : v)         -- when you need to modify in place
```

You don't know the type or it's complex:

```
for (const auto& x : v) -- safe universal choice
```

0.2.39 break and continue

```
for (int i = 0; i < 10; ++i) {
    if (i == 5) {
        break;           // exit the loop immediately
    }
    if (i % 2 == 0) {
        continue;       // skip to the next iteration
    }
    std::cout << i << " "; // prints: 1 3
}
// loop ends because of break when i == 5
```

break and continue only affect the **innermost** enclosing loop. For nested loops, use a function return or a flag:

```
// Method: refactor into a function, use return
bool find_target(const std::vector<std::vector<int>>& grid, int target) {
    for (int row = 0; row < (int)grid.size(); ++row) {
        for (int col = 0; col < (int)grid[row].size(); ++col) {
            if (grid[row][col] == target) {
                return true; // exits both loops
            }
        }
    }
    return false;
}
```

0.2.40 Loop Execution Visualization

For `for (int i = 0; i < 3; ++i) { body; }:`

Start:

```
int i = 0    (initialization -- runs once)
```

Iteration 1:

```
i < 3? YES (0 < 3)
body executes
++i -> i = 1
```

Iteration 2:

```
i < 3? YES (1 < 3)
body executes
++i -> i = 2
```

Iteration 3:

```
i < 3? YES (2 < 3)
body executes
++i -> i = 3
```

Check:

```
i < 3? NO (3 is not < 3)
Loop exits
```

0.2.41 Common Mistakes in This Chapter

0.2.41.1 Mistake 1: Off-By-One Error

The bug:

```
std::vector<int> v = {10, 20, 30};
for (int i = 0; i <= v.size(); ++i) { // <= instead of <
    std::cout << v[i] << "\n";      // accesses v[3] -- does not exist!
}
```

The symptom: Crash, garbage output, or silent memory corruption on the last iteration.

The fix: Use `i < v.size()` (strictly less than). Or better: use range-based `for`, which cannot have this bug.

0.2.41.2 Mistake 2: Forgetting `break` in `switch`

The bug:

```
switch (command) {
    case 'q':
        prepare_to_quit();
        // forgot break -- falls through to 'h'!
    case 'h':
        show_help();
        break;
}
// When command == 'q': prepare_to_quit() runs, THEN show_help() runs
```

The fix: Add break after every case.

0.2.41.3 Mistake 3: Modifying a Container While Iterating Over It

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (int n : v) {
    if (n == 3) v.erase(v.begin() + 2); // modifies v while iterating!
}
// undefined behavior -- the range-based for's internal iterator is now invalid
```

The fix: Collect things to remove first, then remove them after the loop. Or use `std::remove_if` (Chapter 29).

0.2.41.4 Mistake 4: Using = Instead of == in a Condition

The bug:

```
int x = 5;
while (x = 10) { // assigns 10 to x, condition is always true (infinite loop!)
    std::cout << x << "\n";
}
```

The symptom: Infinite loop.

The fix: `while (x == 10)`. Compile with `-Wall` to get a warning.

0.2.42 Exercises

Exercise 4.1 – Predict the output

```
for (int i = 1; i <= 10; ++i) {  
    if (i % 3 == 0) continue;  
    if (i > 7) break;  
    std::cout << i << " ";  
}
```

Answer: 1 2 4 5 7 – skips multiples of 3 (3, 6), breaks when $i > 7$ (stops before 8).

Exercise 4.2 – FizzBuzz

Print numbers 1 to 20. Multiples of 3: print “Fizz”. Multiples of 5: print “Buzz”. Multiples of both 3 and 5: print “FizzBuzz”.

Answer:

```
#include <iostream>  
int main() {  
    for (int i = 1; i <= 20; ++i) {  
        if (i % 15 == 0) std::cout << "FizzBuzz\n";  
        else if (i % 3 == 0) std::cout << "Fizz\n";  
        else if (i % 5 == 0) std::cout << "Buzz\n";  
        else std::cout << i << "\n";  
    }  
}
```

Check % 15 first. If you check % 3 first, 15 would print “Fizz” instead of “FizzBuzz”.

Exercise 4.3 – Sum with while

Use a `while` loop to compute the sum of integers from 1 to 100.

Answer:

```
int sum = 0, i = 1;  
while (i <= 100) {  
    sum += i;  
    ++i;  
}  
std::cout << sum << "\n"; // 5050
```

Exercise 4.4 – Triangle

Print this triangle with nested loops:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Answer:

```
for (int row = 1; row <= 5; ++row) {  
    for (int col = 1; col <= row; ++col) {  
        std::cout << "* ";  
    }  
    std::cout << "\n";  
}
```

Exercise 4.5 – do-while validation

Write a program that asks the user to enter a password. Keep asking until they type “secret”. (Use `std::cin >> password` to read.)

Answer:

```
#include <iostream>  
#include <string>  
  
int main() {  
    std::string password;  
    do {  
        std::cout << "Enter password: ";  
        std::cin >> password;  
    } while (password != "secret");  
    std::cout << "Access granted.\n";  
    return 0;  
}
```

0.2.43 What Happens When You Call a Function

Before the syntax, understand what a function call does to memory. This mental model makes pass-by-value, scope, and the declaration/definition split all click at once.

Your program uses a region of memory called the **call stack**. It operates like a stack of plates: each function call pushes a new **frame** (plate) on top; when the function returns, that frame is popped off and gone.

Each frame holds: - The function's parameters (copies of the arguments) - The function's local variables - The return address (where to continue after returning)

```
int multiply(int a, int b) {
    int result = a * b;
    return result;
}

int square(int x) {
    int s = multiply(x, x);
    return s;
}

int main() {
    int answer = square(5);
    std::cout << answer << "\n"; // 25
}
```

When `square(5)` is called, then `multiply(5, 5)` is called inside `square`:

CALL STACK (top = currently executing function)

+-----+	
multiply's frame	<-- currently executing
a = 5	COPY of square's x
b = 5	COPY of square's x
result = 25	
+-----+	
square's frame	
x = 5	COPY of main's argument 5
s = ?	not yet assigned
+-----+	
main's frame	
answer = ?	not yet assigned
+-----+	

When `multiply` finishes: 1. It returns 25 2. Its entire frame is popped – `a`, `b`, `result` are gone from memory 3. Back in `square`, `s` receives the returned 25 4. `square` returns 25, its frame is popped – `x`, `s` are gone 5. In `main`, `answer` receives 25

Critical insight: `a` and `b` inside `multiply` are COPIES of `x` from `square`. They live at different memory addresses. Changing `a` inside `multiply` has absolutely no effect on `x` in `square`. This is **pass by value**.

0.2.44 Function Syntax

```
# Python -- no return type, no parameter types  
def add(a, b):  
    return a + b
```

```
// C++ -- return type required, parameter types required  
int add(int a, int b) {  
    return a + b;  
}
```

What you must provide: 1. **Return type** before the function name. `int` means the function returns an `int`. 2. **Type for each parameter**. Each parameter is `type name`. 3. A return statement if the return type is not `void`.

```
// Returns nothing:  
void print_banner() {  
    std::cout << "=====\n";  
} // no return needed for void  
  
// Returns bool:  
bool is_palindrome(std::string s) {  
    std::string rev = s;  
    std::reverse(rev.begin(), rev.end());  
    return s == rev;  
}  
  
// Multiple parameters:  
double distance(double x1, double y1, double x2, double y2) {  
    double dx = x2 - x1;  
    double dy = y2 - y1;  
    return std::sqrt(dx*dx + dy*dy);  
}  
  
// No parameters:  
int get_screen_width() {  
    return 1920;  
}
```

0.2.45 Declarations vs Definitions

This is a concept Python does not have. It exists because C++ compiles each `.cpp` file independently, top-to-bottom.

The problem: If you call a function before the compiler has seen it, the compiler does not know: - Does this function exist? - How many parameters does it take? - What types are the parameters? - What type does it return?

Without this information, the compiler cannot type-check the call. It refuses.

A declaration (also called a forward declaration or prototype) gives the compiler the function's signature without the body:

```
int add(int a, int b); // declaration: signature only, ends with semicolon
```

A definition is the full function with its body:

```
int add(int a, int b) { // definition: has the body in braces
    return a + b;
}
```

Every definition is implicitly also a declaration. A declaration without a body is not a definition.

0.2.45.1 The Three Solutions

Problem: function used before it is defined

```
int main() {
    int r = add(3, 4); // ERROR: 'add' not declared yet
}

int add(int a, int b) { return a + b; }
```

Solution 1: Move the definition above the call

```
int add(int a, int b) { return a + b; } // defined first

int main() {
    int r = add(3, 4); // OK -- compiler has seen add
}
```

Works for simple, single-file programs. Becomes unmanageable with circular dependencies (function A calls B, B calls A).

Solution 2: Forward declaration

```
int add(int a, int b); // declaration -- tells compiler the signature

int main() {
    int r = add(3, 4); // OK -- compiler knows signature, trusts you to define it
}

int add(int a, int b) { return a + b; } // definition can appear anywhere after
```

Works in a single file. For multi-file projects, use headers.

Solution 3: Header file (the real solution for multi-file projects)

```
// math.h -- declarations only
#pragma once
int add(int a, int b);
int subtract(int a, int b);

// math.cpp -- definitions
#include "math.h"
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }

// main.cpp -- uses the functions
#include "math.h" // gives main.cpp the declarations it needs
int main() {
    int r = add(3, 4); // compiler checks types via the declaration in math.h
    int s = subtract(10, 3);
}
```

We cover headers fully in Chapter 11.

0.2.46 Pass by Value: What Is Copied and Why It Matters

When you pass a variable to a function, C++ copies it by default. The function gets its own independent copy.

```
void zero_it(int x) {
    x = 0; // modifies the local copy only
    std::cout << "Inside: " << x << "\n";
}

int main() {
    int n = 42;
    zero_it(n);
    std::cout << "Outside: " << n << "\n";
}
```

Output:

Inside: 0

Outside: 42 <-- n was NOT changed

The stack during the call:

```

main's frame:                                zero_it's frame:
+-----+                                +-----+
| n = 42      |                                | x = 42      | <-- a COPY at a different address
| (at 0x7fff1000) |                                | (at 0x7fff0ff0) |
+-----+                                +-----+
                                         |
                                         x = 0 (modifies copy)
                                         |
                                         zero_it returns, frame popped

main's frame:
+-----+
| n = 42      | <-- unchanged
+-----+

```

This applies to every type: `int`, `double`, `std::string`, structs, classes. Passing a `std::string` with 10,000 characters by value copies all 10,000 characters. This is why Chapter 6 (References) and `const&` parameters matter.

0.2.47 Default Parameters

```

# Python
def connect(host, port=8080, timeout=30):
    print(f"Connecting to {host}:{port} (timeout={timeout}s)")

// C++
void connect(std::string host, int port = 8080, int timeout = 30) {
    std::cout << "Connecting to " << host << ":" << port
              << " (timeout=" << timeout << "s)\n";
}

connect("localhost");           // port=8080, timeout=30
connect("localhost", 9000);     // port=9000, timeout=30
connect("localhost", 9000, 60); // port=9000, timeout=60

```

Rules: 1. Default parameters must be at the **end** of the parameter list. 2. You cannot skip a middle argument: `connect("host", , 60)` is illegal. 3. Specify defaults in the **declaration** (header file), not in the definition:

```

// math.h -- put defaults here
void connect(std::string host, int port = 8080, int timeout = 30);

// math.cpp -- no defaults here (would be a redefinition error)
void connect(std::string host, int port, int timeout) {
    // implementation
}

```

0.2.48 Function Overloading

C++ lets you define multiple functions with the same name, as long as they have different parameter types. The compiler picks the right one at each call site.

```
int    abs_val(int x)    { return x < 0 ? -x : x; }
double abs_val(double x) { return x < 0.0 ? -x : x; }
float  abs_val(float x)  { return x < 0.0f ? -x : x; }

abs_val(5);           // calls int version
abs_val(-3.14);        // calls double version
abs_val(-2.0f);        // calls float version (f suffix = float literal)
```

The compiler resolves overloads at compile time using these rules (simplified): 1. **Exact match** – preferred 2. **Trivial conversion** (e.g., non-const to const) 3. **Promotion** (e.g., float to double, short to int) 4. **Standard conversion** (e.g., int to double) 5. **Ambiguous** (two overloads equally good) – compile error

```
void foo(int x)    { std::cout << "int\n"; }
void foo(double x) { std::cout << "double\n"; }

foo(5);           // "int"    -- exact match
foo(5.0);          // "double" -- exact match
foo(5.0f);         // "double" -- float promotes to double (closer than int)
foo('A');          // "int"    -- char promotes to int
// foo(true);     -- ambiguous: bool can promote to int OR double equally well
```

Python does not have overloading. You achieve similar behavior with type checks (`isinstance`) or default parameters. The C++ approach is resolved entirely at compile time with zero runtime cost.

0.2.48.1 What Overloading Is NOT

Overloading is based on **parameter types only**. You cannot overload on return type:

```
int    get() { return 5; }
double get() { return 5.0; } // ERROR: same parameters, only return type differs
```

The compiler uses the call arguments to pick the overload. If two functions take the same arguments, there is no way to distinguish them.

0.2.49 Compiler Messages for Function Errors

Understanding compiler error messages makes debugging much faster. Here are the most common function-related errors:

0.2.49.1 No Matching Function

```
void greet(std::string name) { }
```

```
int main() {  
    greet(42);    // passing int, but greet takes string  
}
```

```
error: no matching function for call to 'greet(int)'  
    note: candidate: 'void greet(std::string)'  
    note:   no known conversion from 'int' to 'std::string'
```

The compiler tells you what it found and why it didn't match.

0.2.49.2 Ambiguous Overload

```
void process(int x) { }  
void process(long x) { }
```

```
process(42);    // 42 is int -- exact match for int version, fine  
process(42L);   // 42L is long -- exact match for long version, fine  
// process(true); -- ambiguous: bool can convert to int OR long
```

```
error: call to 'process' is ambiguous  
    note: candidate: 'void process(int)'  
    note: candidate: 'void process(long)'
```

0.2.49.3 Wrong Number of Arguments

```
void foo(int a, int b) { }
```

```
foo(1, 2, 3);    // too many
```

```
error: too many arguments to function 'void foo(int, int)'  
    note: declared here: void foo(int a, int b)
```

0.2.50 Common Mistakes in This Chapter

0.2.50.1 Mistake 1: Expecting Pass-by-Value to Modify the Original

The bug:

```
void set_to_zero(int x) { x = 0; }

int count = 100;
set_to_zero(count);
std::cout << count;    // 100 -- NOT 0
```

Why it fails: `x` is a copy. Setting `x = 0` changes only the copy.

The fix: Use a reference (Chapter 6): `void set_to_zero(int& x) { x = 0; }`

0.2.50.2 Mistake 2: Missing Return Statement

The bug:

```
int max(int a, int b) {
    if (a > b) return a;
    // forgot: return b;
}
```

The symptom: Compiler warning “control may reach end of non-void function.” Calling `max` when `a <= b` returns garbage.

The fix: All code paths must return a value.

0.2.50.3 Mistake 3: Defining a Function Twice

The bug:

```
int add(int a, int b) { return a + b; }
int add(int a, int b) { return a + b; } // second definition
```

error: redefinition of 'int add(int, int)'

The fix: Declarations can appear multiple times (they're just type information). Definitions can appear exactly once.

0.2.50.4 Mistake 4: Forgetting to Compile the File Containing the Definition

The bug:

```
$ g++ -std=c++23 -o program main.cpp    # forgot greet.cpp
```

undefined reference to `greet(std::string const&)'

The fix: Include all .cpp files that contain definitions you need:

```
$ g++ -std=c++23 -o program main.cpp greet.cpp
```

0.2.51 Exercises

Exercise 5.1 – Declaration or definition?

For each, say whether it is a declaration, a definition, or both:

```
double sqrt(double x);           // (a)
double square(double x) { return x*x; } // (b)
void print();                     // (c)
void print() { std::cout << "!\n"; } // (d)
```

Answer: - (a): declaration only (no body) - (b): definition (has body) – and also implicitly a declaration - (c): declaration only - (d): definition (and implicitly a declaration)

Exercise 5.2 – Stack trace

Draw the call stack when `inner` is executing:

```
int inner(int x)    { return x * 2; }
int middle(int x)   { return inner(x + 1); }
int outer(int x)    { return middle(x + 1); }
int main()          { int r = outer(5); }
```

Answer:

```
inner's frame:  x = 7    (outer 5 -> middle 6 -> inner 7)
middle's frame: x = 6
outer's frame:  x = 5
main's frame:   r = ?
```

inner returns $7 * 2 = 14$. Pops. middle returns 14. Pops. outer returns 14. Pops. main's $r = 14$.

Exercise 5.3 – Overloaded describe

Write three overloaded functions named `describe` that: - Take `int` and print "Integer: N" - Take `double` and print "Double: N.NN" - Take `std::string` and print "String: S (length L)"

Answer:

```
#include <iostream>
#include <string>
#include <iomanip>

void describe(int n) {
    std::cout << "Integer: " << n << "\n";
}

void describe(double d) {
    std::cout << "Double: " << std::fixed << std::setprecision(2) << d << "\n";
}

void describe(std::string s) {
    std::cout << "String: " << s << " (length " << s.size() << ")\n";
}

int main() {
    describe(42);
    describe(3.14159);
    describe(std::string{"hello"});
}
```

Output:

```
Integer: 42
Double: 3.14
String: hello (length 5)
```

Exercise 5.4 – power with default

Write `power (base, exponent=2)` that computes $\text{base}^{\text{exponent}}$ using a loop. It should work for non-negative integer exponents. Verify: `power (3)` returns 9, `power (2, 10)` returns 1024.

Answer:

```
double power(double base, int exponent = 2) {  
    double result = 1.0;  
    for (int i = 0; i < exponent; ++i) {  
        result *= base;  
    }  
    return result;  
}
```

Part I is complete. You now have the foundations: the compilation model, types and memory, operators, control flow, and functions.

Part II begins with the concepts that most distinguish C++ from Python: references, pointers, the stack vs the heap, and const correctness. These are where C++ becomes its own language. Ask to continue.